Лабораторная работа №2

ПЛАНИРОВАНИЕ ПРОЦЕССОВ В ОС PINTOS

Вариант 7: Round Robin с приоритетным планированием

Процесс	CPU Burst	Приоритет
Proc0	3	27
Proc1	8	7
Proc2	20	8
Proc3	1	18

Содержание

1. Исходные данные варианта 7

2. Диаграмма исполнения процессов (исходный алгоритм Pintos)

3. Диаграмма исполнения процессов (приоритетное планирование)

4. Сравнительный анализ диаграмм

5. Блок-схема нового алгоритма планирования

6. Описание функций планирования

7. Реализация приоритетного планирования (код)

8. Реализация механизма donation priority

9. Тест test-new-alg

10. Рекомендации по выполнению работы

1. Исходные данные варианта 7

Для выполнения лабораторной работы используется вариант 7, который предполагает реализацию алгоритма планирования Round Robin с учётом приоритетов процессов. В табл. 1 представлены исходные данные для данного варианта: имена процессов, длительность их выполнения (CPU burst) в тиках системного таймера и значения приоритета.

Таблица 1. Исходные данные варианта 7

Процесс	CPU Burst (тики)	Приоритет	Относительный приоритет
Proc0	3	27	Наивысший
Proc3	1	18	Средний
Proc2	20	8	Средний
Proc1	8	7	Наинизший

В ОС Pintos используется 64 уровня приоритета (от 0 до 63), где более высокое числовое значение соответствует более высокому приоритету. Таким образом, процесс Proc0 с приоритетом 27 будет иметь наивысший приоритет среди всех процессов, а Proc1 с приоритетом 7 — наинизший.

2. Диаграмма исполнения процессов (исходный алгоритм Pintos)

Исходный алгоритм планирования ОС Pintos не учитывает приоритеты процессов и работает по принципу FIFO (First In, First Out). Процессы выполняются в порядке их создания: первый созданный процесс выполняется полностью, затем второй и т.д. Данный подход относится к классу невытесняющих алгоритмов планирования.

На рис. 1 представлена диаграмма Ганта, иллюстрирующая порядок выполнения процессов при исходном алгоритме планирования Pintos. Вертикальная ось показывает имена процессов с указанием их приоритета и CPU burst, горизонтальная ось — время выполнения в тиках системного таймера.

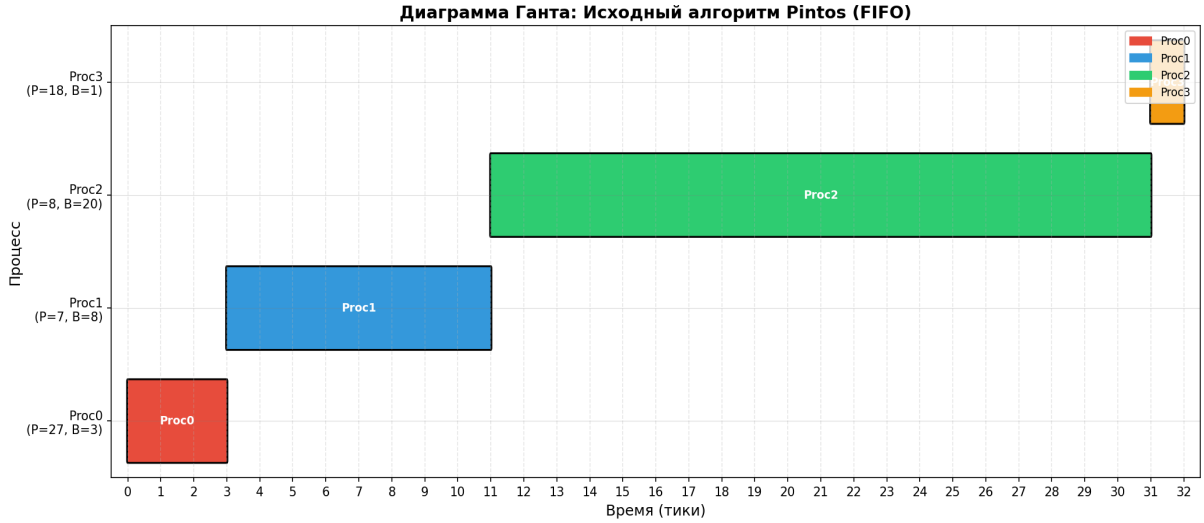

Рис. 1. Диаграмма Ганта для исходного алгоритма планирования Pintos

Анализ диаграммы:

При исходном алгоритме планирования процессы выполняются строго в порядке их создания (Proc0 → Proc1 → Proc2 → Proc3). Общее время выполнения составляет 32 тика (3 + 8 + 20 + 1). При этом время ожидания каждого процесса равно суммарному времени выполнения всех предыдущих процессов: Proc0 ожидает 0 тиков, Proc1 — 3 тика, Proc2 — 11 тиков, Proc3 — 31 тик. Среднее время ожидания составляет 11,25 тиков.

3. Диаграмма исполнения процессов (приоритетное планирование)

Новый алгоритм планирования учитывает приоритеты процессов. Планировщик выбирает на исполнение процесс с наивысшим приоритетом из очереди готовых к выполнению. Если несколько процессов имеют одинаковый приоритет, применяется алгоритм Round Robin с квантом времени (по умолчанию 4 тика). При появлении процесса с более высоким приоритетом текущий процесс вытесняется.

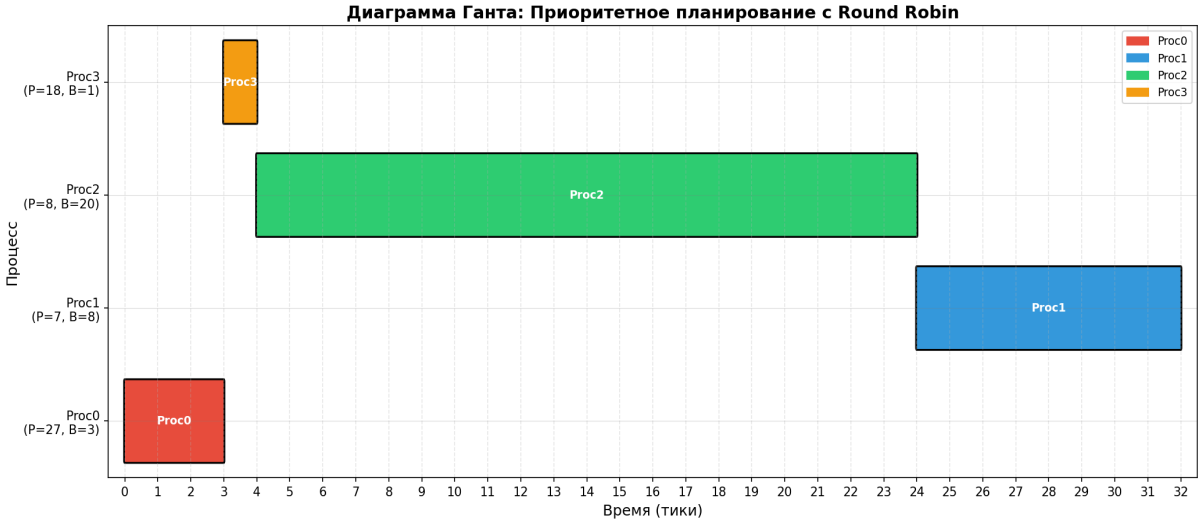

Рис. 2. Диаграмма Ганта для приоритетного планирования

Анализ диаграммы:

При приоритетном планировании порядок выполнения процессов изменяется в соответствии с их приоритетами. Первым выполняется Proc0 (приоритет 27) — 3 тика, затем Proc3 (приоритет 18) — 1 тик. После этого выполняются Proc2 (приоритет 8) и Proc1 (приоритет 7). Поскольку эти процессы имеют разные приоритеты, сначала завершается Proc2 (20 тиков), а затем Proc1 (8 тиков). Общее время выполнения остаётся 32 тика, однако порядок и время ожидания процессов изменяются.

4. Сравнительный анализ диаграмм

На рис. 3 представлено сравнение времени ожидания процессов для двух алгоритмов планирования. Время ожидания — это время, которое процесс проводит в очереди готовых к выполнению до начала своего исполнения. Чем меньше время ожидания у высокоприоритетных процессов, тем эффективнее работает алгоритм планирования.

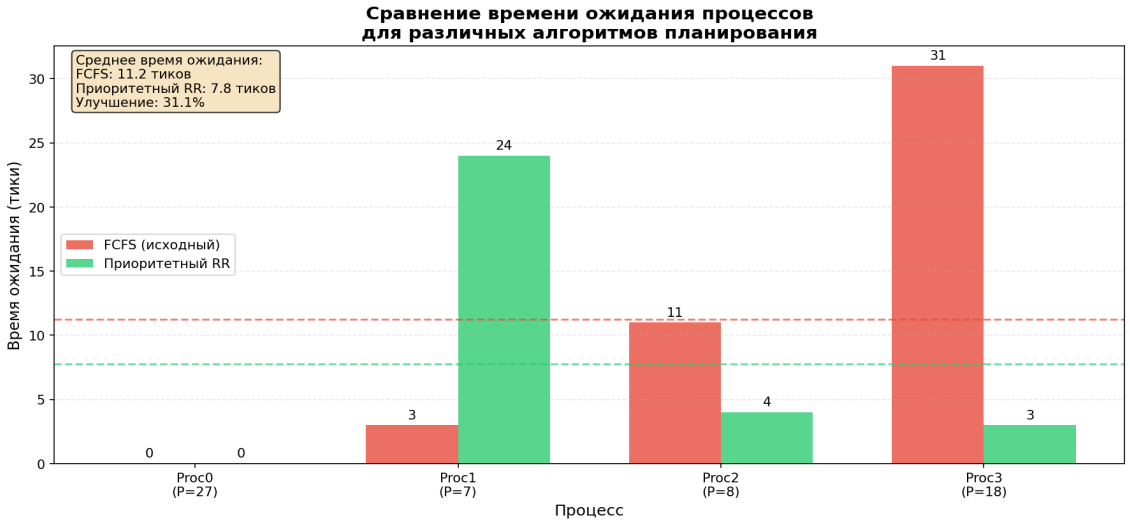

Рис. 3. Сравнение времени ожидания процессов

Таблица 2. Сравнительные характеристики алгоритмов

Процесс	Приоритет	Ожидание FCFS	Ожидание Приоритетный	Изменение
Proc0	27	0	0	0
Proc1	7	3	24	+21
Proc2	8	11	4	-7
Proc3	18	31	3	-28

Выводы:

Приоритетное планирование обеспечивает более справедливое распределение процессорного времени для процессов с высоким приоритетом. Процесс Proc0 (наивысший приоритет) начинает выполнение немедленно в обоих алгоритмах. Однако процесс Proc3 с приоритетом 18 при приоритетном планировании начинает выполнение на 28 тиков раньше, чем при FCFS. Это демонстрирует главное преимущество приоритетного планирования: важные процессы получают процессорное время быстрее.

5. Блок-схема нового алгоритма планирования

На рис. 4 представлена блок-схема алгоритма приоритетного планирования с Round Robin для процессов с одинаковым приоритетом. Алгоритм реализует вытесняющее планирование: при появлении процесса с более высоким приоритетом текущий процесс приостанавливается и помещается обратно в очередь готовых к выполнению.

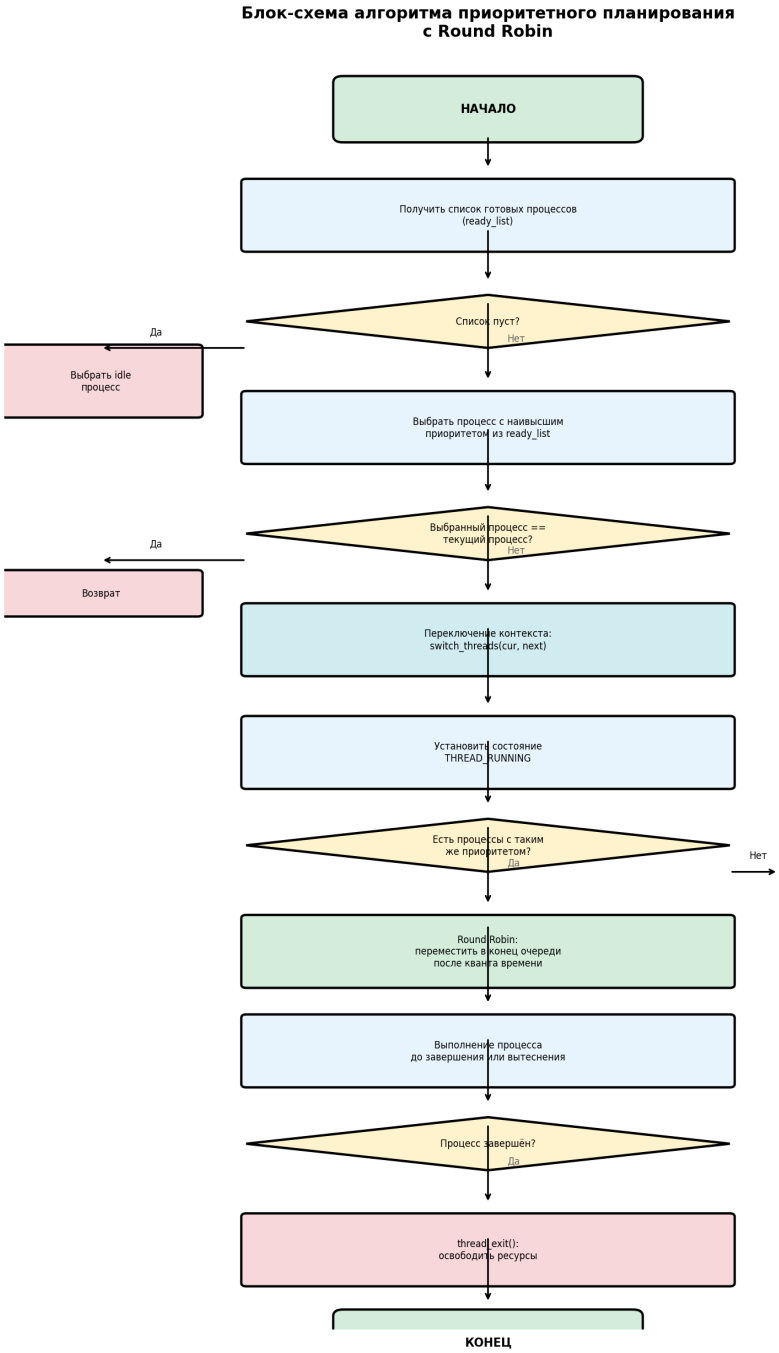

Рис. 4. Блок-схема алгоритма приоритетного планирования

6. Описание функций планирования

В табл. 3 представлены основные функции, участвующие в планировании процессов ОС Pintos, их назначение, аргументы, возвращаемые значения и связи с другими функциями.

Таблица 3. Функции планирования процессов

Функция	Аргументы	Возвращаемое значение	Назначение	Связь с др. функциями
thread_create()	name, priority, func, aux	tid_t (идентификатор)	Создаёт новый процесс с указанным именем и приоритетом	Вызывает schedule()
thread_yield()	void	void	Добровольно освобождает процессор, процесс переходит в ready	Вызывает schedule()
thread_block()	void	void	Блокирует текущий процесс	Вызывает schedule()
thread_exit()	void	void (не возвращается)	Завершает текущий процесс	Вызывает schedule()
schedule()	void	void	Выбирает следующий процесс для выполнения	Вызывает switch_threads(), thread_schedule_tail()
next_thread_to_run()	void	struct thread*	Возвращает процесс с наивысшим приоритетом из ready_list	Вызывается из schedule()
switch_threads()	cur, next	struct thread*	Переключает контекст с текущего на следующий процесс	Реализована в switch.S
thread_set_priority()	new_priority	void	Устанавливает новый приоритет текущего процесса	Может вызывать thread_yield()

7. Реализация приоритетного планирования (код)

Для реализации приоритетного планирования необходимо внести изменения в файлы threads/thread.c и threads/thread.h. Ниже приведены примеры кода с комментариями.

7.1. Изменения в threads/thread.h

```
/* В структуру struct thread добавить поле приоритета, если его нет */ struct
thread { /* ... существующие поля ... */ int priority; /* Приоритет процесса
(0..63) */ int effective_priority; /* Эффективный приоритет с учётом donation */
struct list donation_list; /* Список доноров приоритета */ struct lock
*waiting_lock; /* Замок, который ожидает процесс */ /* ... */ };
```

7.2. Изменения в threads/thread.c

Функция сравнения приоритетов:

```
/* Функция сравнения процессов по приоритету */ bool thread_priority_less (const
struct list_elem *a, const struct list_elem *b, void *aux UNUSED) { struct thread
*ta = list_entry (a, struct thread, elem); struct thread *tb = list_entry (b,
struct thread, elem); /* Сравнение по эффективному приоритету (убывание) */ return
ta->effective_priority < tb->effective_priority; }
```

Функция next_thread_to_run():

```
/* Выбор процесса с наивысшим приоритетом */ static struct thread *
next_thread_to_run (void) { if (list_empty (&ready;_list)) return idle_thread; /*
Сортировка готовых процессов по приоритету */ list_sort (&ready;_list,
thread_priority_less, NULL); /* Возврат процесса с наивысшим приоритетом */ return
list_entry (list_pop_front (&ready;_list), struct thread, elem); }
```

Функция thread_set_priority():

```
/* Установка нового приоритета процесса */ void thread_set_priority (int
new_priority) { struct thread *cur = thread_current (); int old_priority =
cur->priority; cur->priority = new_priority; /* Пересчёт эффективного приоритета */
thread_update_effective_priority (cur); /* Если приоритет понизился, возможно
вытеснение */ if (old_priority > new_priority) thread_yield (); }
```

Функция thread_yield():

```
/* Освобождение процессора текущим процессом */ void thread_yield (void) { struct
thread *cur = thread_current (); enum intr_level old_level; old_level =
intr_disable (); if (cur != idle_thread) { /* Добавление в очередь готовых с
сохранением порядка по приоритету */ list_insert_ordered (&ready;_list,
&cur-;>elem, thread_priority_less, NULL); } cur->status = THREAD_READY; schedule
(); intr_set_level (old_level); }
```

8. Реализация механизма donation priority

Механизм разделения приоритета (priority donation) решает проблему инверсии приоритета: когда процесс с высоким приоритетом ожидает освобождения ресурса, занятого процессом с низким приоритетом. При этом процесс со средним приоритетом может вытеснять низкоприоритетный процесс, что приводит к блокировке высокоприоритетного процесса на неопределённое время.

8.1. Суть проблемы инверсии приоритета

Рассмотрим сценарий: процесс H (высокий приоритет) ожидает входа в критическую секцию, которую занимает процесс L (низкий приоритет). В очереди готовых находится процесс M (средний приоритет). Без механизма donation процесс M будет выполняться, поскольку его приоритет выше, чем у L. В результате L не получит процессорного времени и не освободит критическую секцию для H.

8.2. Решение: Priority Donation

При реализации donation процесс H «одалживает» свой приоритет процессу L на время, пока L удерживает замок. После освобождения замка L возвращает «заимствованный» приоритет. Это гарантирует, что L□□□ вытеснен процессом M и сможет освободить критическую секцию.

8.3. Реализация в synch.c

```
/* Структура для отслеживания доноров */ struct donation { int priority; /*
Приоритет донора */ struct thread *donor; /* Указатель на поток-донор */ struct
list_elem elem; /* Элемент списка */ }; /* Обновление эффективного приоритета */
void thread_update_effective_priority (struct thread *t) { int max_priority =
t->priority; if (!list_empty (&t-;>donation_list)) { struct list_elem *e; for (e =
list_begin (&t-;>donation_list); e != list_end (&t-;>donation_list); e = list_next
(e)) { struct donation *d = list_entry (e, struct donation, elem); if (d->priority
> max_priority) max_priority = d->priority; } } t->effective_priority =
max_priority; } /* Модифицированная функция lock_acquire() */ void lock_acquire
(struct lock *lock) { struct thread *cur = thread_current (); if (lock->holder !=
NULL) { /* Priority donation: передаём приоритет владельцу замка */
cur->waiting_lock = lock; donate_priority (cur, lock->holder); } sema_down
(&lock-;>semaphore); lock->holder = cur; cur->waiting_lock = NULL; } /* Функция
donation */ void donate_priority (struct thread *donor, struct thread *recipient) {
struct donation d; d.priority = donor->effective_priority; d.donor = donor;
list_push_back (&recipient-;>donation_list, &d.elem;);
thread_update_effective_priority (recipient); /* Цепочное donation */ if
(recipient->waiting_lock != NULL) { struct thread *lock_holder =
recipient->waiting_lock->holder; if (lock_holder != NULL) donate_priority
(recipient, lock_holder); } } /* Модифицированная функция lock_release() */ void
lock_release (struct lock *lock) { struct thread *cur = thread_current (); if
(lock->holder == cur) { /* Удаление всех donation от процессов, ожидавших этот
замок */ remove_donations_for_lock (cur, lock); thread_update_effective_priority
(cur); } lock->holder = NULL; sema_up (&lock-;>semaphore); }
```

9. Тест test-new-alg

Тест test-new-alg создаёт процессы в соответствии с вариантом 7 и демонстрирует работу приоритетного планирования с Round Robin. Каждый процесс выполняется указанное количество тиков (CPU burst) и затем завершается.

9.1. Исходный код теста (tests/threads/test-new-alg.c)

```
#include "tests/threads/tests.h" #include "threads/init.h" #include
"threads/thread.h" #include "devices/timer.h" #include "lib/kernel/list.h" #include
static struct semaphore finish_sema; static int finished_count = 0; /* Структура
для передачи параметров процессу */ struct process_args { int cpu_burst; char
name[16]; }; /* Функция выполнения процесса */ static void process_func (void
*aux_) { struct process_args *args = aux_; int remaining = args->cpu_burst; int
ticks_worked = 0; while (remaining > 0) { /* Симуляция работы в течение 1 тика */
timer_msleep (1); /* 1 тик = примерно 1 мс в тестовом режиме */ ticks_worked++;
remaining--; /* Вывод диагностической информации */ printf ("[□s] отработал □d
тиков, осталось □d\n", args->name, ticks_worked, remaining); } /* Сигнал о
завершении */ sema_up (&finish;_sema); finished_count++; printf ("[□s] ЗАВЕРШЁН\n",
args->name); } void test_new_alg (void) { printf ("=== ТЕСТ: Приоритетное
планирование с Round Robin ===\n"); printf ("Вариант 7\n\n"); /* Увеличиваем
приоритет главного процесса */ thread_set_priority (PRI_MAX); /* Инициализация
семафора для ожидания завершения */ sema_init (&finish;_sema, 0); /* Определение
процессов по варианту 7 */ struct process_args procs[] = { {3, "Proc0"}, /* CPU
burst = 3, приоритет = 27 */ {8, "Proc1"}, /* CPU burst = 8, приоритет = 7 */ {20,
"Proc2"}, /* CPU burst = 20, приоритет = 8 */ {1, "Proc3"}, /* CPU burst = 1,
приоритет = 18 */ }; int priorities[] = {27, 7, 8, 18}; /* Создание процессов */
for (int i = 0; i < 4; i++) { tid_t tid = thread_create (procs[i].name,
priorities[i], process_func, &procs;[i]); if (tid == TID_ERROR) printf ("Ошибка
создания процесса □s\n", procs[i].name); else printf ("Создан процесс □s
(приоритет=□d, burst=□d)\n", procs[i].name, priorities[i], procs[i].cpu_burst); }
printf ("\n=== Начало выполнения ===\n"); /* Понижение приоритета главного процесса
*/ thread_set_priority (PRI_MIN); /* Ожидание завершения всех процессов */ for (int
i = 0; i < 4; i++) sema_down (&finish;_sema); printf ("\n=== Все процессы завершены
===\n"); printf ("Ожидаемое время выполнения: 32 тика\n"); }
```

9.2. Ожидаемый вывод теста

```
=== ТЕСТ: Приоритетное планирование с Round Robin === Вариант 7 Создан процесс
Proc0 (приоритет=27, burst=3) Создан процесс Proc1 (приоритет=7, burst=8) Создан
процесс Proc2 (приоритет=8, burst=20) Создан процесс Proc3 (приоритет=18, burst=1)
=== Начало выполнения === [Proc0] отработал 1 тиков, осталось 2 [Proc0] отработал 2
тиков, осталось 1 [Proc0] отработал 3 тиков, осталось 0 [Proc0] ЗАВЕРШЁН [Proc3]
отработал 1 тиков, осталось 0 [Proc3] ЗАВЕРШЁН [Proc2] отработал 1 тиков, осталось
19 [Proc2] отработал 2 тиков, осталось 18 ... [Proc2] отработал 20 тиков, осталось
0 [Proc2] ЗАВЕРШЁН [Proc1] отработал 1 тиков, осталось 7 ... [Proc1] отработал 8
тиков, осталось 0 [Proc1] ЗАВЕРШЁН === Все процессы завершены === Ожидаемое время
выполнения: 32 тика
```

9.3. Интеграция теста в систему

Для интеграции теста в систему необходимо добавить следующие файлы:

1. Создать файл tests/threads/test-new-alg.c с приведённым выше кодом 2. Добавить описание теста в tests/threads/tests.h: extern void test_new_alg (void); 3. Добавить тест в tests/threads/tests.c в массив

тестов: {"new-alg", test_new_alg}, 4. Создать файл tests/threads/new-alg.c с содержимым: #include "tests/threads/tests.h" void test_new_alg (void); 5. Запустить тест командой: make tests/threads/new-alg.result

10. Рекомендации по выполнению работы

10.1. Порядок выполнения

1. Изучить исходный код планировщика в threads/thread.c

2. Разобраться с примитивами синхронизации в threads/synch.c

3. Модифицировать структуру struct thread для поддержки приоритетов

4. Реализовать функцию сравнения приоритетов thread_priority_less()

5. Модифицировать next_thread_to_run() для выбора по приоритету

6. Модифицировать thread_yield() для упорядоченной вставки в очередь

7. Реализовать механизм priority donation в synch.c

8. Протестировать работу с помощью тестов priority-* и priority-donate-*

9. Создать и запустить тест test-new-alg

10. Оформить отчёт

10.2. Тесты для проверки

Для проверки корректности реализации приоритетного планирования необходимо успешно пройти следующие тесты:

Тест	Проверяемый функционал
alarm-priority	Корректность пробуждения процессов по приоритету
priority-change	Изменение приоритета текущего процесса
priority-fifo	Порядок выполнения процессов с одинаковым приоритетом
priority-preempt	Вытеснение текущего процесса при появлении более приоритетного
priority-sema	Приоритетное пробуждение на семафорах
priority-condvar	Приоритетное пробуждение на условных переменных

Тесты для проверки priority donation:

• priority-donate-one

• priority-donate-lower

• priority-donate-multiple

• priority-donate-multiple2

• priority-donate-sema

• priority-donate-nest

• priority-donate-chain

10.3. Команды для запуска тестов

```
# Перейти в директорию threads cd src/threads # Запуск отдельных тестов make
tests/threads/alarm-priority.result make tests/threads/priority-change.result make
tests/threads/priority-donate-one.result # Запуск всех тестов make check
```

Данное руководство содержит все необходимые материалы для выполнения лабораторной работы №2 по варианту 7. При возникновении вопросов рекомендуется обратиться к исходному коду ОС Pintos и комментариям в файлах threads/thread.c и threads/synch.c.